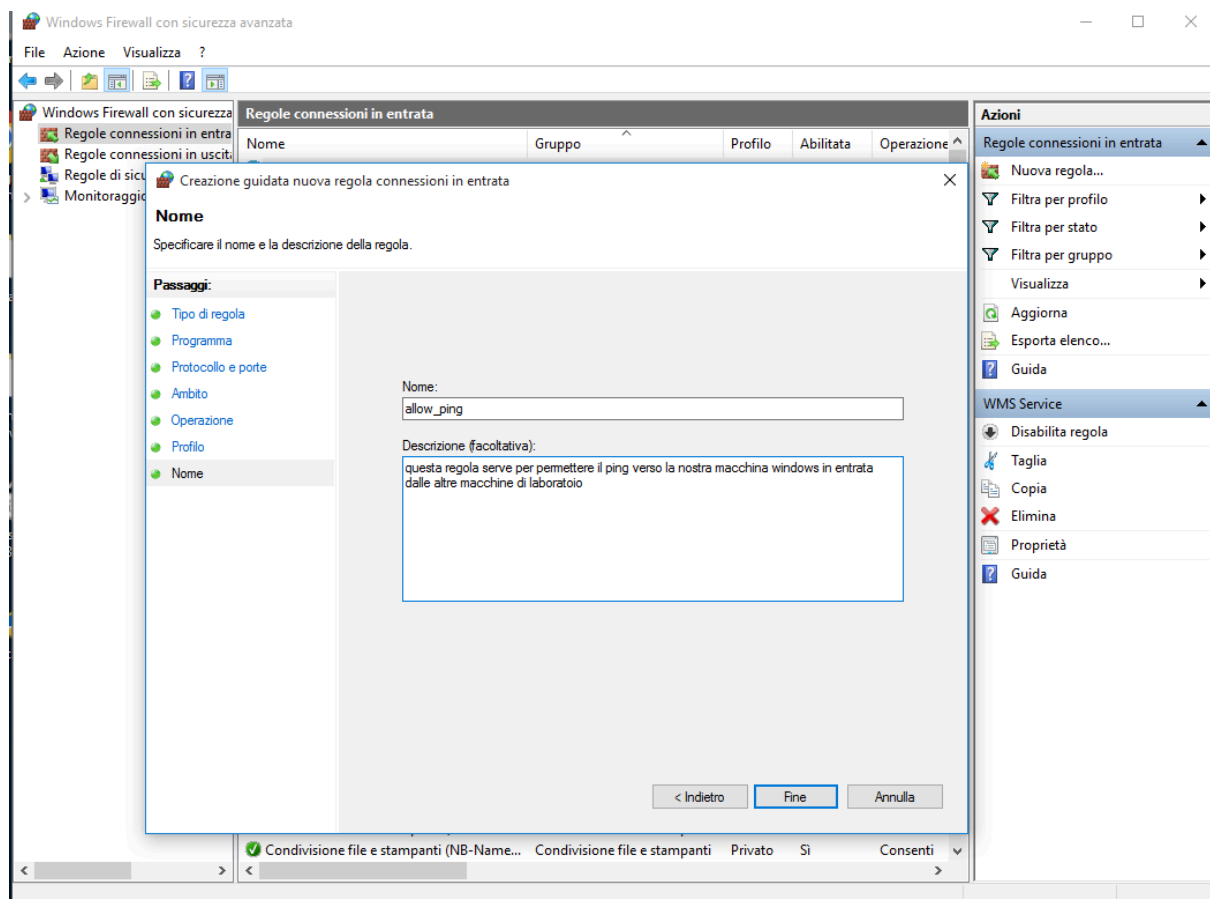


Consegna 09/01

Parte 1

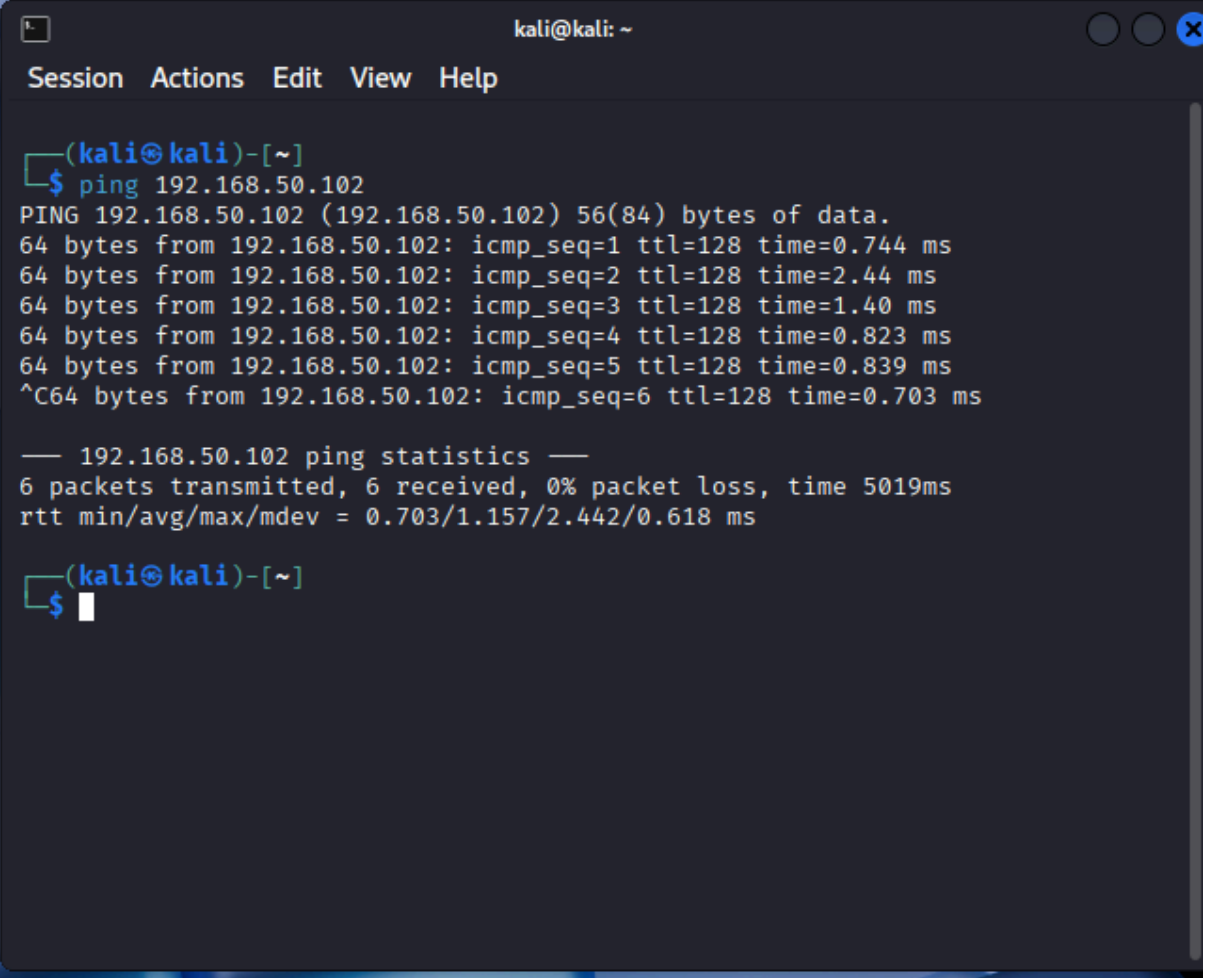
Configurare policy per permettere il ping da macchina Linux a macchina Windows nel nostro laboratorio (Windows firewall)

in questo esercizio andiamo a configurare l'ICMP consentito in entrata , con una regola del firewall windows



Tramite le impostazioni avanzate del firewall andiamo a impostare il tipo di regola , il programma ,che rimarranno bianchi e il protocollo che sarà ICMPv4 su tutte le porte , creando questa regola diamo la possibilità nella nostra rete locale (Virtualbox)di poter arrivare alla macchina tramite ping.

per verificarlo andiamo in kali e pinghiamo il nostro Windows metasploitable su terminale.

A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: ~'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a user prompt '(kali@kali)-[~]' followed by the command '\$ ping 192.168.50.102'. The output shows six successful ping responses with varying times (0.744 ms to 2.44 ms). After the sixth response, the user presses Ctrl-C, indicated by '^C'. The terminal then displays 'ping statistics' for 192.168.50.102, showing 6 packets transmitted, 6 received, 0% packet loss, and a total time of 5019ms. The statistics also show an rtt of 0.703/1.157/2.442/0.618 ms. The prompt returns to '(kali@kali)-[~]' followed by '\$' and a cursor.

```
kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ ping 192.168.50.102
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data:
64 bytes from 192.168.50.102: icmp_seq=1 ttl=128 time=0.744 ms
64 bytes from 192.168.50.102: icmp_seq=2 ttl=128 time=2.44 ms
64 bytes from 192.168.50.102: icmp_seq=3 ttl=128 time=1.40 ms
64 bytes from 192.168.50.102: icmp_seq=4 ttl=128 time=0.823 ms
64 bytes from 192.168.50.102: icmp_seq=5 ttl=128 time=0.839 ms
^C64 bytes from 192.168.50.102: icmp_seq=6 ttl=128 time=0.703 ms

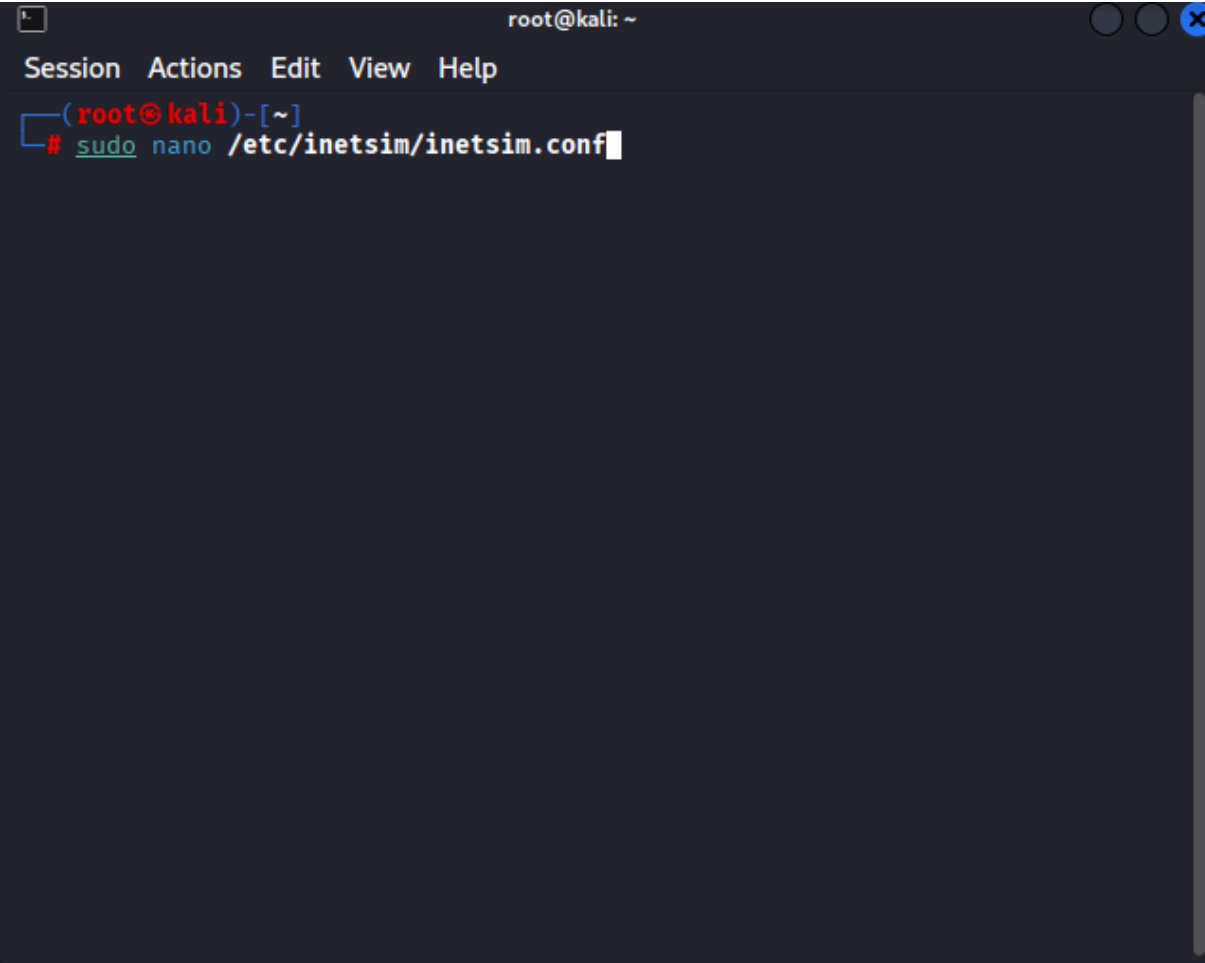
— 192.168.50.102 ping statistics —
6 packets transmitted, 6 received, 0% packet loss, time 5019ms
rtt min/avg/max/mdev = 0.703/1.157/2.442/0.618 ms

(kali@kali)-[~]
$
```

come possiamo vedere tutti i pacchetti sono stati ricevuti . (n.b. ho dovuto fare clear sulla mia kali in quanto il firewall su windows era disattivato quindi il ping mi era già passato e il comando è rimasto in memoria...)dopo aver fatto questa prova abbiamo disattivato momentaneamente la regola.

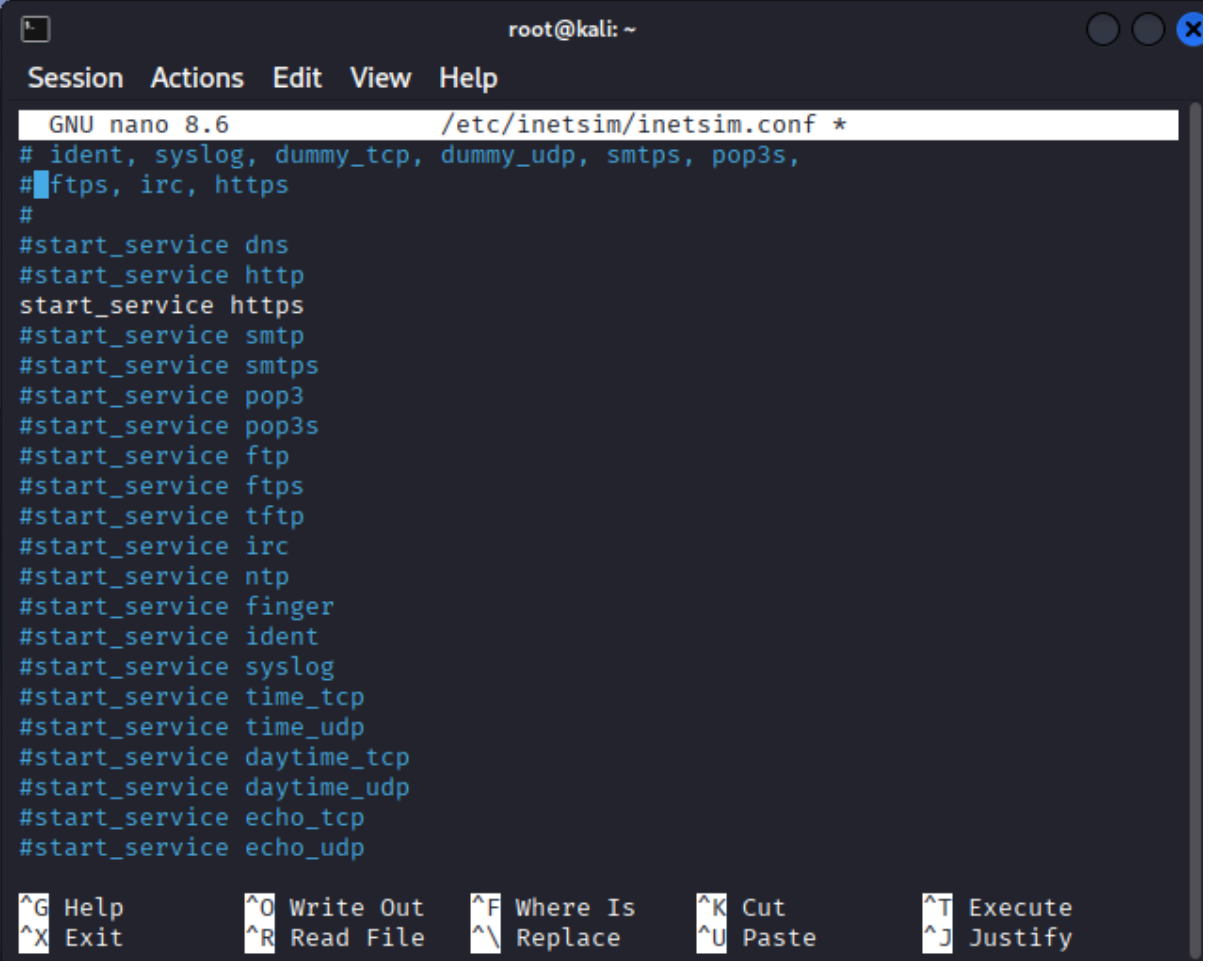
ES2 Utilizzo dell'utility InetSim per l'emulazione di servizi Internet

Andando su Kali possiamo simulare servizi internet tramite diverse applicazioni , una di queste è InetSim .



```
root@kali: ~  
Session Actions Edit View Help  
(root@kali)~  
# sudo nano /etc/inetsim/inetsim.conf
```

come primo passo andiamo a configurare inetsim con il solo servizio https attivo.



```
root@kali: ~  
Session Actions Edit View Help  
GNU nano 8.6 /etc/inetsim/inetsim.conf *  
# ident, syslog, dummy_tcp, dummy_udp, smtps, pop3s,  
# ftps, irc, https  
#  
#start_service dns  
#start_service http  
start_service https  
#start_service smtp  
#start_service smtps  
#start_service pop3  
#start_service pop3s  
#start_service ftp  
#start_service ftps  
#start_service tftp  
#start_service irc  
#start_service ntp  
#start_service finger  
#start_service ident  
#start_service syslog  
#start_service time_tcp  
#start_service time_udp  
#start_service daytime_tcp  
#start_service daytime_udp  
#start_service echo_tcp  
#start_service echo_udp  
  
^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute  
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify
```

di default sono attivi tutti i servizi , per disattivarli e tenere solo quelli che vogliamo utilizziamo # per disattivarli

```
root@kali: ~
Session Actions Edit View Help
(root@kali)-[~]
# sudo inetsim
\INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Main logfile '/var/log/inetsim/main.log' does not exist. Trying to create it.
..
Main logfile '/var/log/inetsim/main.log' successfully created.
Sub logfile '/var/log/inetsim/service.log' does not exist. Trying to create i
t ...
Sub logfile '/var/log/inetsim/service.log' successfully created.
Debug logfile '/var/log/inetsim/debug.log' does not exist. Trying to create i
t ...
Debug logfile '/var/log/inetsim/debug.log' successfully created.
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
== INetSim main process started (PID 21177) ==
Session ID:      21177
Listening on:    127.0.0.1
Real Date/Time:  2026-01-09 08:11:52
Fake Date/Time: 2026-01-09 08:11:52 (Delta: 0 seconds)
Forking services...
  * https_443_tcp - started (PID 21187)
done.
Simulation running.
```

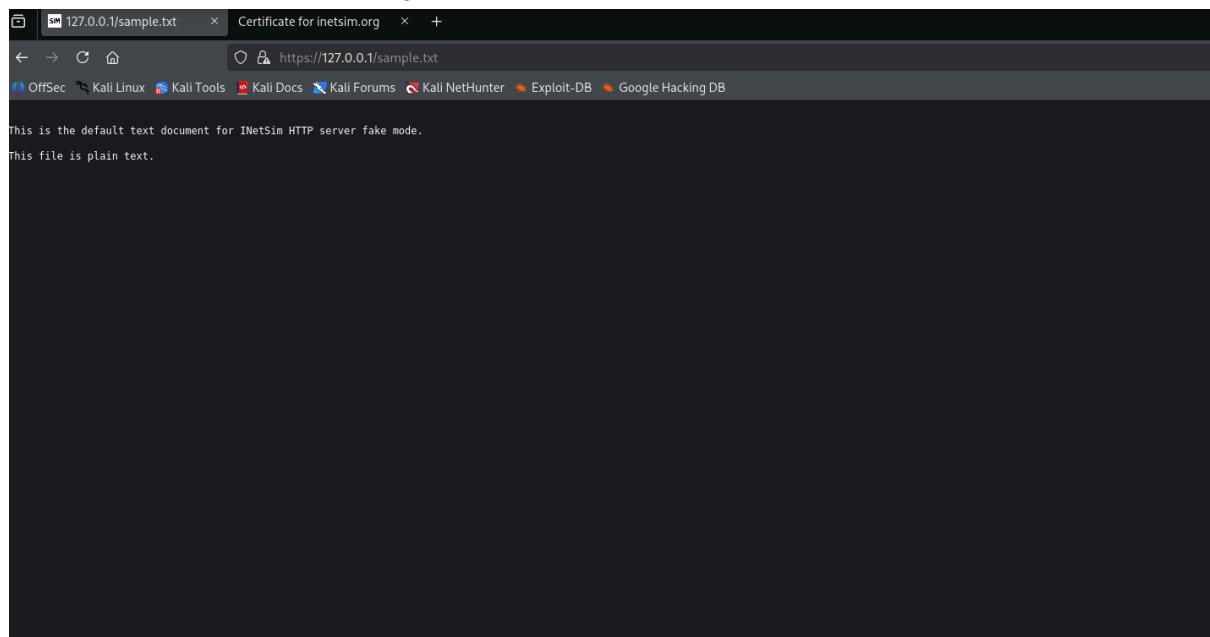
dopo aver salvato uscendo il file conf di inetsim lo lanciamo per avere il nostro servizio https che di default troveremo all'indirizzo 127.0.0.1 .

Tramite web browser verifichiamo che tutto funzioni correttamente



This is the default HTML page for INetSim HTTP server fake mode.
This file is an HTML document.

come possiamo vedere inetsim sta funzionando da ed è in esecuzione, per sicurezza passiamo a vedere un'altra pagina.



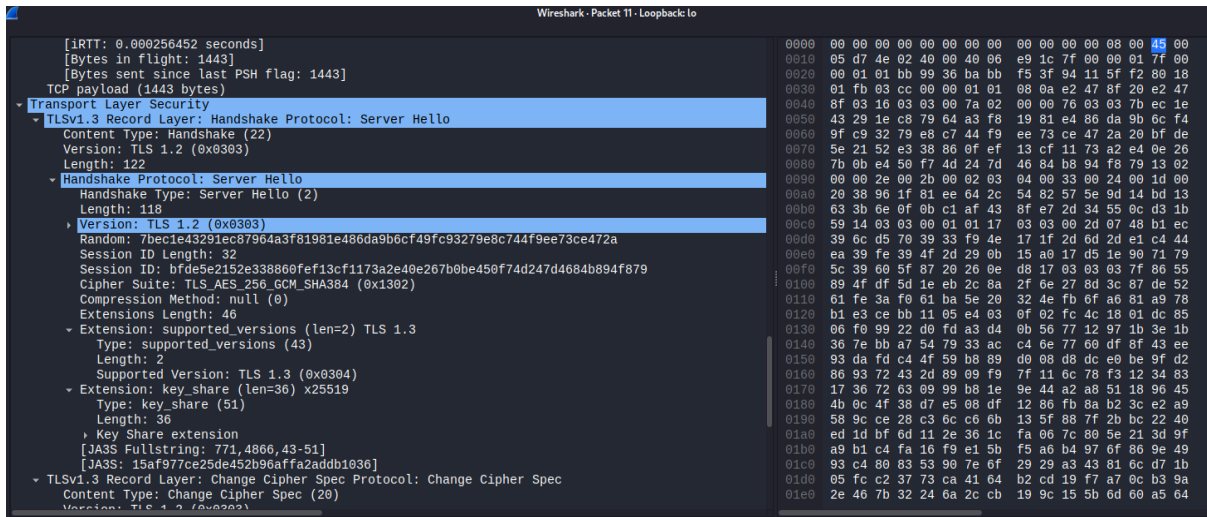
Anche il sample.txt si carica correttamente, ora con questo ora possiamo svolgere l'esercizio 3 ovvero la cattura dei pacchetti wireshark

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	41324 → 443 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3793410916 TSecr=0 WS=128
2	0.000013646	127.0.0.1	127.0.0.1	TCP	74	443 → 41324 [SYN, ACK] Seq=0 Ack=1 Win=65493 Len=0 MSS=65495 SACK_PERM TSval=3793410916 TSecr=3793410916 WS=128
3	0.000023198	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3793410916 TSecr=3793410916
4	0.000050581	127.0.0.1	127.0.0.1	TLSv1.3	785	Client Hello
5	0.000061025	127.0.0.1	127.0.0.1	TCP	66	443 → 41324 [ACK] Seq=1 Ack=640 Win=64896 Len=0 TSval=3793410917 TSecr=3793410917
6	0.047645140	127.0.0.1	127.0.0.1	TLSv1.3	1509	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
7	0.047664820	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=640 Ack=1444 Win=76928 Len=0 TSval=3793410964 TSecr=3793410964
8	0.102457990	127.0.0.1	127.0.0.1	TLSv1.3	146	Change Cipher Spec, Application Data
9	0.102534941	127.0.0.1	127.0.0.1	TCP	66	443 → 41324 [ACK] Seq=1444 Ack=720 Win=64896 Len=0 TSval=3793411019 TSecr=3793411019
10	0.102918838	127.0.0.1	127.0.0.1	TLSv1.3	321	Application Data
11	0.103081977	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=720 Ack=1699 Win=77696 Len=0 TSval=3793411019 TSecr=3793411019
12	0.103095449	127.0.0.1	127.0.0.1	TLSv1.3	321	Application Data
13	0.103098925	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=720 Ack=1954 Win=77568 Len=0 TSval=3793411019 TSecr=3793411019
25	1.619303838	127.0.0.1	127.0.0.1	TLSv1.3	521	Application Data
26	1.657819533	127.0.0.1	127.0.0.1	TLSv1.3	239	Application Data
27	1.657912671	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=1175 Ack=2127 Win=77696 Len=0 TSval=3793412574 TSecr=3793412574
28	1.657998210	127.0.0.1	127.0.0.1	TLSv1.3	346	Application Data
29	1.657998255	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [ACK] Seq=1175 Ack=2407 Win=77568 Len=0 TSval=3793412574 TSecr=3793412574
30	1.659166142	127.0.0.1	127.0.0.1	TLSv1.3	99	Application Data
31	1.659288580	127.0.0.1	127.0.0.1	TCP	66	41324 → 443 [FIN, ACK] Seq=1199 Ack=2407 Win=77696 Len=0 TSval=3793412576 TSecr=3793412574
32	1.660606266	127.0.0.1	127.0.0.1	TLSv1.3	99	Application Data
33	1.660639907	127.0.0.1	127.0.0.1	TCP	54	41324 → 443 [RST] Seq=1200 Win=0 Len=0

inserendo i filtri di cattura in loopback ("in localhost" molto virgolettato) andiamo a fare la cattura dei pacchetti dando di nuovo l'invio al nostro fake web server. come possiamo vedere

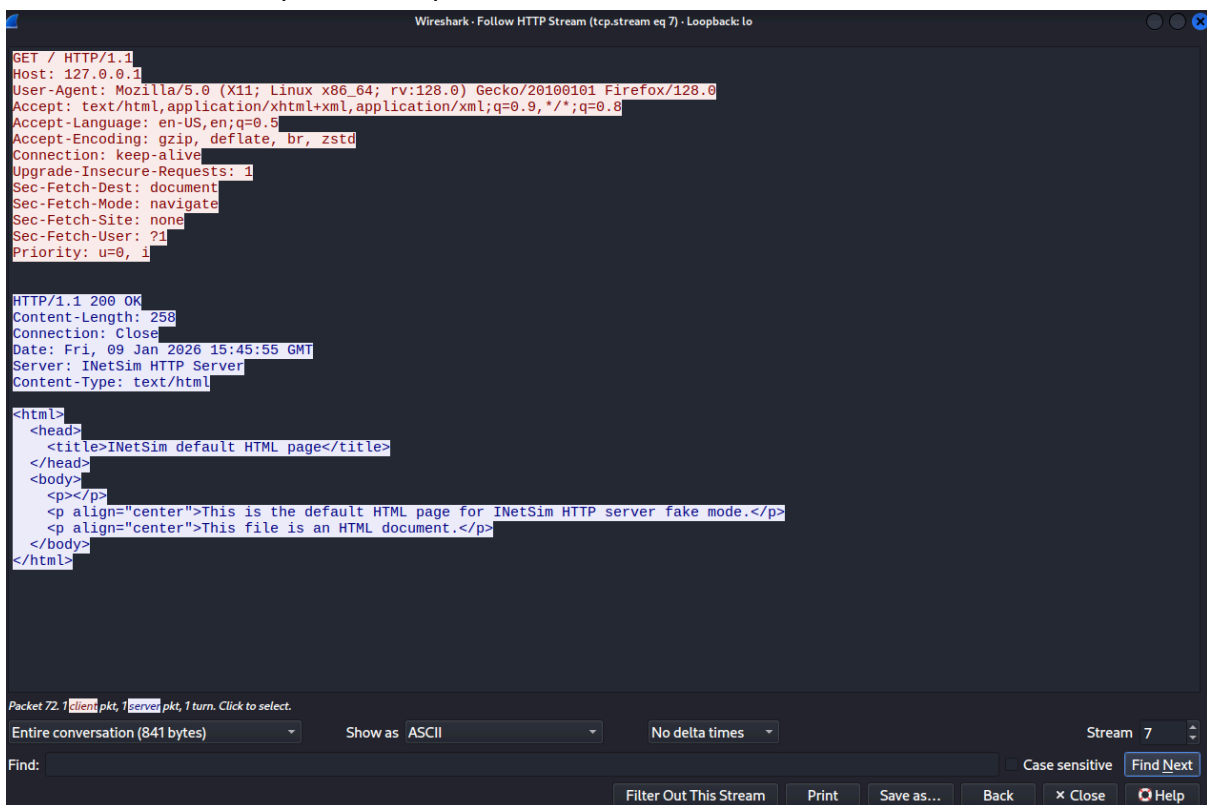
c'è una sequenzialità di syn e ack che il web server fa con se stesso per vedere se tutti i servizi sono attivi e funzionanti. il protocollo di cattura dei pacchetti fa sì che vengano mostrati con filtro i pacchetti tcp dove possiamo vedere che la destinazione è esso stesso. la considerazione primaria che mi viene da fare analizzando i pacchetti è che il payload,

sembra abbastanza grande per due linee html



La cosa che mi sorprende un po' dopo aver controllato un po' i pacchetti è che come si può trovare parte della chiave della cifratura nel pubblica.

Successivamente come prova ho visto seguito un flusso html e visualizzato così la pagina ovviamente non in https ma in http



ovviamente possiamo vedere come wireshark sotto questo punto di vista e con una buona conoscenza di networking possa essere davvero uno strumento utilissimo per capire dove e come viaggiano i pacchetti e come sono suddivisi dall'header al payload interpretandone i contenuti.